

Documento de Casos de Teste — Processamento de Pagamentos

Módulo: src/pagamento.py
Arquivo de testes: tests/test_pagamento.py
Framework: pytest, pytest-mock, requests

Funções testadas

Função	Descrição
processar_compra(usuario_id, dados_cartao, valor)	Coordena o processamento de um pagamento via gateway externo, retornando o status da transação (sucesso, recusa ou erro de comunicação).

Casos de Teste

CT-01 — Compra Aprovada (O Caminho Feliz)

Campo	Detalhe
ID	CT-01
Função testada	processar_compra
Objetivo	Verificar se o sistema processa corretamente uma compra aprovada pelo gateway de pagamento
Entradas	usuario_id = 1, dados_cartao = {"numero": "4000123456789010", "cvv": "123"}, valor = 150.0
Resultado esperado	Retorno da string "Sucesso! Transação PAG-123456 confirmada." e chamada correta da função dependente
Técnica	Mock de retorno via return_value, igualdade direta (assert ==) e verificação de chamada (assert_called_once_with)
Tipo	Caso normal (caminho feliz)

Por que usar `mock_gateway.assert_called_once_with`?

Além de garantir que a função principal retorne o texto certo, é vital atestar que os dados sensíveis (o cartão e o valor) foram repassados exatamente como esperado para o gateway de terceiros, sem alterações indesejadas.

```
def test_processar_compra_sucesso(mocker):
    """Testa se o sistema processa corretamente uma compra aprovada pelo gateway."""

    mock_gateway = mocker.patch('src.pagamento.enviar_para_gateway')

    mock_gateway.return_value = {
        "status": "aprovado",
        "transacao_id": "PAG-123456"
    }

    dados_cartao = {"numero": "4000123456789010", "cvv": "123"}
    resultado = processar_compra(usuario_id=1, dados_cartao=dados_cartao, valor=150.0)

    assert resultado == "Sucesso! Transação PAG-123456 confirmada."
    mock_gateway.assert_called_once_with(dados_cartao, 150.0)
```

CT-02 — Cliente Sem Limite (Mock de Retorno Específico)

Campo	Detalhe
ID	CT-02
Função testada	processar_compra
Objetivo	Verificar o comportamento do sistema quando o cartão é recusado pelo gateway por falta de limite
Entradas	usuario_id = 1, dados_cartao = {"numero": "4000123456789010", "cvv": "123"}, valor = 9999.0
Resultado esperado	Retorno da string "Pagamento recusado. Motivo: Cartão sem limite disponível."
Técnica	Mock de dicionário de retorno (status recusado) e igualdade direta (assert ==)
Tipo	Caso alternativo (regra de negócio)

```
def test_processar_compra_recusada_sem_limite(mocker):
    """Testa o comportamento do sistema quando o cartão é recusado por falta de limite.
    """

    mock_gateway = mocker.patch('src.pagamento.enviar_para_gateway')

    mock_gateway.return_value = {
        "status": "recusado",
        "motivo": "Cartão sem limite disponível"
    }

    dados_cartao = {"numero": "4000123456789010", "cvv": "123"}
    resultado = processar_compra(usuario_id=1, dados_cartao=dados_cartao, valor=9999.0)

    assert resultado == "Pagamento recusado. Motivo: Cartão sem limite disponível."
```

CT-03 — Timeout na Black Friday (Mock de Exceção)

Campo	Detalhe
ID	CT-03
Função testada	processar_compra
Objetivo	Verificar se o sistema lida de forma segura quando a API do gateway fica inacessível ou lenta, retornando Timeout
Entradas	usuario_id = 1, dados_cartao = {"numero": "4000123456789010", "cvv": "123"}, valor = 200.0
Resultado esperado	Retorno da string amigável de erro de comunicação ("Tempo de resposta esgotado. Verifique sua fatura antes de tentar de novo.")
Técnica	mocker.patch em bibliotecas nativas (requests.post) acoplado com side_effect para injeção de exceção
Tipo	Caso de erro / resiliência de sistema

Por que usar side_effect em vez de return_value?

Diferente de um retorno JSON convencional de erro de API, um Timeout quebra a execução do código (levanta uma exceção). O side_effect serve justamente para simular cenários onde funções falham em vez de concluir suas rotinas.

```
def test_processar_compra_timeout_na_black_friday(mocker):
    """Testa se o sistema lida com segurança quando o gateway cai por Timeout."""

    mock_post = mocker.patch('src.pagamento.requests.post')

    mock_post.side_effect = requests.exceptions.Timeout()

    dados_cartao = {"numero": "4000123456789010", "cvv": "123"}
    resultado = processar_compra(usuario_id=1, dados_cartao=dados_cartao, valor=200.0)

    assert resultado == "Tempo de resposta esgotado. Verifique sua fatura antes de tentar de novo."
```

Resumo das técnicas utilizadas

Técnica	Quando usar
<code>mocker.patch('caminho_absoluto')</code>	Para interceptar chamadas a funções dependentes, APIs ou componentes difíceis de testar em ambiente isolado.
<code>mock.return_value = {...}</code>	Simular a devolução correta de dados (como um JSON ou dicionário) de uma API de terceiros.
<code>mock.side_effect = Excecao()</code>	Testar blocos try/except forçando uma quebra artificial no comportamento do sistema.
<code>mock.assert_called_once_with(args)</code>	Validar se parâmetros corretos e exatos foram enviados para a função dependente ao menos e no máximo uma vez.

Como executar os testes

A partir da raiz do projeto:

```
# Rodar todos os testes de pagamento com detalhes de cada caso
pytest tests/test_pagamento.py -v
# Rodar um teste específico de falha
pytest tests/test_pagamento.py::test_processar_compra_timeout_na_black_friday
```

Resultado esperado ao rodar `pytest -v`

```
tests/test_pagamento.py::test_processar_compra_sucesso PASSED
tests/test_pagamento.py::test_processar_compra_recusada_sem_limite PASSED
tests/test_pagamento.py::test_processar_compra_timeout_na_black_friday PASSED
3 passed in 0.XXs
```